

WEEK-3

Task 1: To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop

```
root@LAPTOP-RJFD8EBT:~# nano task1.c
root@LAPTOP-RJFD8EBT:~# cat task1.c
#include <stdio.h>
#include <string.h>

int main()
{
    char input[100];

    while (1)
    {
        printf("myshell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
        {
            break;
        }

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
        {
            printf("Exiting shell...\n");
            break;
        }

        if (strlen(input) == 0)
        {
            continue;
        }

        printf("You entered: %s\n", input);
    }

    return 0;
}
root@LAPTOP-RJFD8EBT:~# gcc -Wall -Wextra task1.c -o task1
root@LAPTOP-RJFD8EBT:~# ls
File1.txt      OSSP          process      task1.c      task3      task4.c
"File1.txt'"  command_executor process.c    task2      task3.c    tasks1
root@LAPTOP-RJFD8EBT:~# gcc -Wall -Wextra task1.c -o task1
root@LAPTOP-RJFD8EBT:~# ls
File1.txt      OSSP          process      task1.c      task3      task4.c
"File1.txt'"  command_executor process.c    task2      task3.c    tasks1
File2.txt      command_executor.c task1      task2.c      task4      week3
root@LAPTOP-RJFD8EBT:~# ./task1
myshell> hello
You entered: hello
myshell> Operating System
You entered: Operating System
myshell> enter
You entered: enter
myshell> exit
Exiting shell...
root@LAPTOP-RJFD8EBT:~# |
```

Task 2: Using the following system calls, copy the content from File1.txt to File2.txt.

1. **Open()**
2. **Read()**
3. **Write()**
4. **Close()**

```
root@LAPTOP-RJFD8EBT:~# nano File1.txt
root@LAPTOP-RJFD8EBT:~# cat File1.txt
This is Operating Systems Lab.
This file is used to demonstrate system calls.
The contents will be copied from File1.txt to File2.txt.
root@LAPTOP-RJFD8EBT:~# nano task2.c
root@LAPTOP-RJFD8EBT:~# cat task2.c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main()
{
    int fd1, fd2;
    char buffer[100];
    ssize_t n;

    // Open File1.txt for reading
    fd1 = open("File1.txt", O_RDONLY);

    if (fd1 == -1)
    {
        perror("Error opening File1.txt");
        return 1;
    }

    // Create/Open File2.txt for writing
    fd2 = open("File2.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);

    if (fd2 == -1)
    {
        perror("Error opening File2.txt");
        close(fd1);
        return 1;
    }

    // Read from File1.txt and write to File2.txt
    while ((n = read(fd1, buffer, sizeof(buffer))) > 0)
    {
        if (write(fd2, buffer, n) != n)
        {
            perror("Error writing to File2.txt");
            close(fd1);
            close(fd2);
            return 1;
        }
    }
}
```

```

// Read from File1.txt and write to File2.txt
while ((n = read(fd1, buffer, sizeof(buffer))) > 0)
{
    if (write(fd2, buffer, n) != n)
    {
        perror("Error writing to File2.txt");
        close(fd1);
        close(fd2);
        return 1;
    }
}

if (n == -1)
{
    perror("Error reading File1.txt");
}
else
{
    printf("File copied successfully.\n");
}

// Close both files
close(fd1);
close(fd2);

return 0;
}
root@LAPTOP-RJFD8EBT:~# gcc -Wall -Wextra task2.c -o task2
root@LAPTOP-RJFD8EBT:~# ls
File1.txt      OSSP          process       task1.c       task3         task4.c
"File1.txt"    command_executor process.c     task2         task3.c       tasks1
File2.txt      command_executor.c task1        task2.c       task4         week3
root@LAPTOP-RJFD8EBT:~# ./task2
File copied successfully.
root@LAPTOP-RJFD8EBT:~# cat File2.txt
This is Operating Systems Lab.
This file is used to demonstrate system calls.
The contents will be copied from File1.txt to File2.txt.
root@LAPTOP-RJFD8EBT:~# cmp File1.txt File2.txt
root@LAPTOP-RJFD8EBT:~# |

```

Task 3: To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction.

```
root@LAPTOP-RJFD8EBT:~# nano task3.c
root@LAPTOP-RJFD8EBT:~# cat task3.c
#include <stdio.h>
#include <unistd.h>
#include <termios.h>
#include <string.h>

#define MAX 100

int main()
{
    char buffer[MAX];
    char ch;
    int index;

    struct termios oldt, newt;

    // Save current terminal settings
    tcgetattr(STDIN_FILENO, &oldt);

    // Copy old settings
    newt = oldt;

    // Disable canonical mode and echo
    newt.c_lflag &= ~(ICANON | ECHO);

    // Apply new terminal settings
    tcsetattr(STDIN_FILENO, TCSANOW, &newt);

    // Repeat forever
    while (1)
    {
        // Display prompt
        write(STDOUT_FILENO, "\nmysh> ", 7);

        index = 0;

        // Read characters one by one
        while (1)
        {
            read(STDIN_FILENO, &ch, 1);

            // Enter key
            if (ch == '\n' || ch == '\r')
            {
                buffer[index] = '\0';

                write(STDOUT_FILENO, "\n", 1);
```

```

        // Backspace
        else if ((ch == 127 || ch == 8) && index > 0)
        {
            index--;

            write(STDOUT_FILENO, "\b \b", 3);
        }

        // Normal character
        else if (index < MAX - 1)
        {
            buffer[index] = ch;
            index++;

            write(STDOUT_FILENO, &ch, 1);
        }
    }

    // Check for exit command
    if (strcmp(buffer, "exit") == 0)
    {
        break;
    }

    // Display command
    write(STDOUT_FILENO, "Command: ", 9);
    write(STDOUT_FILENO, buffer, strlen(buffer));
    write(STDOUT_FILENO, "\n", 1);
}

// Restore original terminal settings
tcsetattr(STDIN_FILENO, TCSANOW, &oldt);

return 0;
}
root@LAPTOP-RJFD8EBT:~# gcc -Wall -Wextra task3.c -o task3
root@LAPTOP-RJFD8EBT:~# ./task3

mysh> hello
Command: hello

mysh> Operating System
Command: Operating System

mysh> ./task3
Command: ./task3

mysh> exit
root@LAPTOP-RJFD8EBT:~# |

```

Task 4: To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Update Input Buffer, Test Recall Functionality

```

root@LAPTOP-RJFD8EBT:~# nano task4.c
root@LAPTOP-RJFD8EBT:~# ^C
root@LAPTOP-RJFD8EBT:~# cat task4.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <termios.h>
#include <sys/wait.h>

#define MAX 100
#define HISTORY_SIZE 10

char history[HISTORY_SIZE][MAX];
int history_count = 0;
int history_index = 0;

void save_history(char *command)
{
    if (strlen(command) == 0)
        return;

    if (history_count < HISTORY_SIZE)
    {
        strcpy(history[history_count], command);
        history_count++;
    }
    else
    {
        for (int i = 0; i < HISTORY_SIZE - 1; i++)
            strcpy(history[i], history[i + 1]);

        strcpy(history[HISTORY_SIZE - 1], command);
    }

    history_index = history_count;
}

void clear_line(int length)
{
    for (int i = 0; i < length; i++)
        write(STDOUT_FILENO, "\b \b", 3);
}

int main()
{
    struct termios oldt, newt;
    char buffer[MAX];
    char ch;

```

```

int main()
{
    struct termios oldt, newt;
    char buffer[MAX];
    char ch;
    int index;

    tcgetattr(STDIN_FILENO, &oldt);

    newt = oldt;
    newt.c_lflag &= ~(ICANON | ECHO);

    tcsetattr(STDIN_FILENO, TCSANOW, &newt);

    while (1)
    {
        write(STDOUT_FILENO, "mysh> ", 6);

        index = 0;
        history_index = history_count;

        while (1)
        {
            read(STDIN_FILENO, &ch, 1);

            /* Enter */
            if (ch == '\n' || ch == '\r')
            {
                buffer[index] = '\0';
                write(STDOUT_FILENO, "\n", 1);
                break;
            }

            /* Backspace */
            else if ((ch == 127 || ch == 8) && index > 0)
            {
                index--;
                write(STDOUT_FILENO, "\b \b", 3);
            }

            /* Escape sequence */
            else if (ch == 27)
            {
                char seq[2];

                read(STDIN_FILENO, &seq[0], 1);
                read(STDIN_FILENO, &seq[1], 1);
            }
        }
    }
}

```

```

/* Up Arrow */
if (seq[0] == '[' && seq[1] == 'A')
{
    if (history_index > 0)
    {
        history_index--;

        clear_line(index);

        strcpy(buffer, history[history_index]);
        index = strlen(buffer);

        write(STDOUT_FILENO, buffer, index);
    }
}

/* Down Arrow */
else if (seq[0] == '[' && seq[1] == 'B')
{
    if (history_index < history_count - 1)
    {
        history_index++;

        clear_line(index);

        strcpy(buffer, history[history_index]);
        index = strlen(buffer);

        write(STDOUT_FILENO, buffer, index);
    }
    else if (history_index == history_count - 1)
    {
        history_index = history_count;

        clear_line(index);

        index = 0;
        buffer[0] = '\0';
    }
}

/* Normal character */
else if (index < MAX - 1)
{
    buffer[index++] = ch;
    write(STDOUT_FILENO, &ch, 1);
}

```

```

    }

    /* Exit */
    if (strcmp(buffer, "exit") == 0)
        break;

    /* Store command in history */
    save_history(buffer);

    /* Tokenize command */
    char *args[MAX];
    int argc = 0;

    char *token = strtok(buffer, " ");

    while (token != NULL && argc < MAX - 1)
    {
        args[argc++] = token;
        token = strtok(NULL, " ");
    }

    args[argc] = NULL;

    if (argc == 0)
        continue;

    /* Create child process */
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
    }
    else if (pid == 0)
    {
        /* Child */
        execvp(args[0], args);

        perror("execvp");
        exit(1);
    }
    else
    {
        /* Parent */
        wait(NULL);
    }
}

/* Restore terminal settings */

```

```

}

/* Restore terminal settings */
tcsetattr(STDIN_FILENO, TCSANOW, &oldt);

return 0;
}

root@LAPTOP-RJFD8EBT:~# gcc -Wall -Wextra task4.c -o task4
root@LAPTOP-RJFD8EBT:~# ls -l task4
-rwxr-xr-x 1 root root 16696 Aug 12 07:10 task4
root@LAPTOP-RJFD8EBT:~# ./task4
mysh> ls
File1.txt      OSSP      process    task1.c    task3      task4.c
"File1.txt"    command_executor process.c    task2      task3.c    tasks1
File2.txt      command_executor.c task1      task2.c    task4      week3
mysh> pwd
/root
mysh> date
Wed Aug 12 07:11:31 UTC 2026
mysh> whoami
root
mysh> ls
File1.txt      OSSP      process    task1.c    task3      task4.c
"File1.txt"    command_executor process.c    task2      task3.c    tasks1
File2.txt      command_executor.c task1      task2.c    task4      week3
mysh> pwd
/root
mysh> date
Wed Aug 12 07:12:13 UTC 2026
mysh> pwd
/root
mysh> ls
File1.txt      OSSP      process    task1.c    task3      task4.c
"File1.txt"    command_executor process.c    task2      task3.c    tasks1
File2.txt      command_executor.c task1      task2.c    task4      week3
mysh> echo Hello
Hello
mysh> exit
root@LAPTOP-RJFD8EBT:~#

```

Task 5: To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind.

```
root@LAPTOP-RJFD8EBT:~/task5# nano task5.c
root@LAPTOP-RJFD8EBT:~/task5# ls
task5.c
root@LAPTOP-RJFD8EBT:~/task5# cat task5.c
#include <stdio.h>
#include <stdlib.h>

/* Linked List Node */
struct Node {
    int data;
    struct Node *next;
};

/* 1. Dynamic Buffer Allocation */
void dynamicBuffer()
{
    int size;

    printf("\n--- Dynamic Buffer Allocation ---\n");

    printf("Enter buffer size: ");
    scanf("%d", &size);

    char *buffer = (char *)malloc(size * sizeof(char));

    if (buffer == NULL) {
        printf("Memory allocation failed!\n");
        return;
    }

    printf("Enter a string: ");
    getchar();

    fgets(buffer, size, stdin);

    printf("Stored string: %s", buffer);

    free(buffer);
    buffer = NULL;
}
```

```
    printf("Buffer memory released successfully.\n");
}

/* 2. Resize Array using realloc */
void resizeArray()
{
    int n, newSize;

    printf("\n--- Resize Array ---\n");

    printf("Enter initial array size: ");
    scanf("%d", &n);

    int *arr = (int *)malloc(n * sizeof(int));

    if (arr == NULL) {
        printf("Memory allocation failed!\n");
        return;
    }

    printf("Enter %d elements:\n", n);

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Enter new array size: ");
    scanf("%d", &newSize);

    int *temp = (int *)realloc(arr, newSize * sizeof(int));

    if (temp == NULL) {
        printf("Memory reallocation failed!\n");
        free(arr);
        return;
    }

    arr = temp;

    if (newSize > n) {
        printf("Enter %d additional elements:\n", newSize - n);

        for (int i = n; i < newSize; i++) {
            scanf("%d", &arr[i]);
        }
    }

    printf("Array after resizing:\n");
}
```

```

    arr = NULL;

    printf("Array memory released successfully.\n");
}

/* 3. Prevent Buffer Overflow */
void preventBufferOverflow()
{
    char buffer[20];

    printf("\n--- Prevent Buffer Overflow ---\n");

    printf("Enter a string (maximum 19 characters): ");

    scanf("%19s", buffer);

    printf("Safe input: %s\n", buffer);
}

/* Insert Node */
struct Node *insertNode(struct Node *head, int value)
{
    struct Node *newNode =
        (struct Node *)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        return head;
    }

    newNode->data = value;
    newNode->next = head;

    return newNode;
}

/* Display Linked List */
void displayList(struct Node *head)
{
    struct Node *temp = head;

    printf("Linked List: ");

    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }

    printf("NULL\n");
}

```

```

while (1) {

    printf("\n=====\\n");
    printf("          OS TASK 5 - MEMORY\\n");
    printf("=====\\n");

    printf("1. Dynamic Buffer Allocation\\n");
    printf("2. Resize Array using realloc\\n");
    printf("3. Prevent Buffer Overflow\\n");
    printf("4. Dynamic Linked List\\n");
    printf("5. Exit\\n");

    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice) {

        case 1:
            dynamicBuffer();
            break;

        case 2:
            resizeArray();
            break;

        case 3:
            preventBufferOverflow();
            break;

        case 4:
            linkedList();
            break;

        case 5:
            printf("\\nProgram terminated.\\n");
            return 0;

        default:
            printf("Invalid choice!\\n");
    }
}

return 0;
}

```

root@LAPTOP-RJFD8EBT:~/task5# gcc -Wall -Wextra -g task5.c -o task5
 root@LAPTOP-RJFD8EBT:~/task5# ls -l
 total 32
 -wxr-xr-x 1 root root 20704 Aug 17 05:07 task5

```

}
root@LAPTOP-RJFD8EBT:~/task5# gcc -Wall -Wextra -g task5.c -o task5
root@LAPTOP-RJFD8EBT:~/task5# ls -l
total 32
-rwxr-xr-x 1 root root 20704 Aug 17 05:07 task5
-rw-r--r-- 1 root root 4360 Aug 17 05:06 task5.c
root@LAPTOP-RJFD8EBT:~/task5# 1
1: command not found
root@LAPTOP-RJFD8EBT:~/task5# 30
30: command not found
root@LAPTOP-RJFD8EBT:~/task5# ./task5

=====
          OS TASK 5 - MEMORY
=====
1. Dynamic Buffer Allocation
2. Resize Array using realloc
3. Prevent Buffer Overflow
4. Dynamic Linked List
5. Exit
Enter your choice: 1

--- Dynamic Buffer Allocation ---
Enter buffer size: 30
Enter a string: OperatingSystem
Stored string: OperatingSystem
Buffer memory released successfully.

=====
          OS TASK 5 - MEMORY
=====
1. Dynamic Buffer Allocation
2. Resize Array using realloc
3. Prevent Buffer Overflow
4. Dynamic Linked List
5. Exit
Enter your choice: 5

Program terminated.
root@LAPTOP-RJFD8EBT:~/task5#

```

```

}

/* Free Linked List */
void freeList(struct Node *head)
{
    struct Node *temp;

    while (head != NULL) {
        temp = head;
        head = head->next;

        free(temp);
    }
}

/* 4. Dynamic Linked List */
void linkedList()
{
    struct Node *head = NULL;

    int n;
    int value;

    printf("\n--- Dynamic Linked List ---\n");

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter value: ");
        scanf("%d", &value);

        head = insertNode(head, value);
    }

    displayList(head);

    freeList(head);

    head = NULL;

    printf("Linked list memory released successfully.\n");
}

/* Main Function */
int main()
{

```